

RTL8762E Proximity Application Design Spec

V1.0
2021/08/26

修订历史

日期	版本	修改	作者
2021/08/26	V0.1	初稿，由 RTL8762D 文档 继承	

目 录

修订历史	2
目 录	3
图目录	4
表目录	5
1. 概述.....	6
1.1. 器件清单.....	6
1.2. 系统需求.....	6
1.3. 术语定义.....	7
2. 硬件设计.....	8
2.1. 电路设计	8
2.2. 引脚定义.....	8
3. 防丢器的软件概述.....	9
4. 防丢器 IO 初始化和处理.....	10
4.1. IO 初始化	10
4.2. IO 处理	11
4.2.1. 按键的处理.....	11
4.2.2. IO 逻辑的实现	12
5. 广播.....	13
6. GATT 相关 Service 和 Characteristic.....	15
6.1. Immediately Alert Service	15
6.2. Link Loss Alert Service	18
6.3. Tx Power Service	21
6.4. Battery Service	22
6.5. Device Information Service	25
6.6. Key Notification Service	34
7. 服务的初始化和注册回调函数.....	38
8. DLPS	39
8.1. DLPS 概述.....	39
8.2. DLPS 使能和配置.....	39
8.3. DLPS 的条件和唤醒源	40
9. 参考文献.....	42

图目录

图 1.1 防丢器使用场景.....	6
图 3.1 防丢器应用系统框图.....	9
图 4.1 IO 的初始化流程.....	10
图 4.2 蓝牙状态迁移.....	12

表目录

表 6.1 包含的 Service 以及 UUID 列表.....	15
表 6.2 Immediate Alert Service Characteristic 列表.....	15
表 6.3 Alert Level Characteristic Value Format	15
表 6.4 Alert Level Enumerations	16
表 6.5 Alert Level Enumerations	17
表 6.6 Link Loss Service Characteristic 列表.....	18
表 6.7 Alert Level Characteristic Value Format	18
表 6.8 Alert Level Enumerations	18
表 6.9 Tx Power Service Characteristic 列表.....	21
表 6.10 Tx Power Level Characteristic Value Format	21
表 6.11 Battery Service Characteristic 列表	22
表 6.12 Battery Level Characteristic Value Format	22
表 6.13 Device Information Service Characteristic 列表	25
表 6.14 Device Information Characteristic Value Format	25
表 6.15 System ID Characteristic Value Format	26
表 6.16 IEEE 11073-20601 Regulatory Certification Data List Characteristic Value Format ..	26
表 6.17 PnP ID Characteristic Value Format.....	26
表 6.18 Vendor ID Enumerations	27
表 6.19 Key Notification Service Characteristic 列表.....	34
表 6.20 Set Alert Time Characteristic Value Format	34
表 6.21 Key Value Characteristic Value Format.....	34

1. 概述

防丢器主要用于防止手机、钱包、钥匙、行李等贵重物品丢失和寻找丢失的物品。移动终端和防丢器处于连接状态时，双方均可发起寻找功能，即实现声光报警。当移动终端和防丢器距离较远时，双方也会同时发出报警提醒。



图 1.1 防丢器使用场景

1.1. 器件清单

1. Bee Evaluation Board 母版
2. RTL8762E 子板
3. 蜂鸣器

1.2. 系统需求

PC 端需要下载和安装的工具：

1. Keil MDK-ARM 5.12
2. SEGGER's J-Link tools
3. RTL8762E SDK
4. RTL8762x Flash programming algorithm

移动设备端需要安装的软件工具：

1. RtkPxp

1.3. 术语定义

1. 对测端：安装有防丢器应用(RtkPxp APP)的 Android 或者 IOS 移动设备；
2. 防丢器：基于 RTL8762E 开发的防丢器应用；
3. DPLS：Deep Low Power State, 超低功耗睡眠状态。

2. 硬件设计

2.1. 电路设计

电路部分可以用 RTL8762E EVB 进行模拟，具体请参照 RTL8762E EVB SCH。使用 EVB 上的 KEY2 模拟按键，使用 EVB 上的 LED0 模拟 LED，使用 EVB 上的 LED1 代替 BEEP 。

2.2. 引脚定义

RTL8762E 子板：

LED	P0_1
BEEP	P0_2
KEY	P2_4

3. 防丢器的软件概述

防丢器通过 IO Driver 和 BT 的交互，完成特定的功能，其主要功能架构如图 3.1 所示。

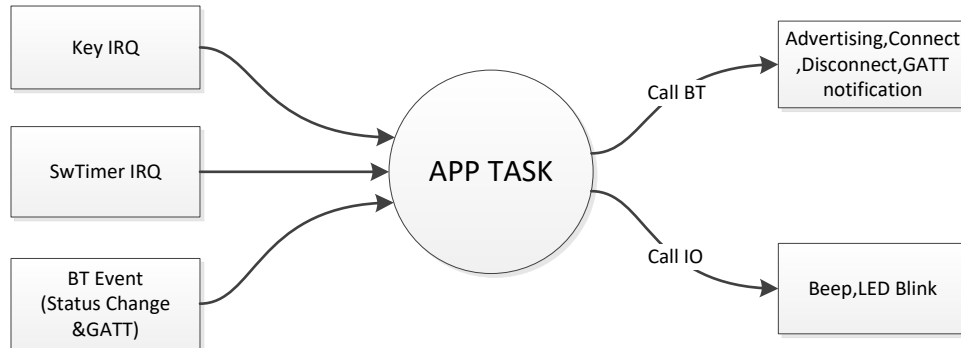


图 3.1 防丢器应用系统框图

系统初始化之后，App task 开始运行，该任务首先创建应用层消息队列、注册 Upper Stack 的应用层回调函数、初始化应用层的外围接口，然后通过不断地查询消息队列完成消息的处理。Upper stack，IO ISR，Timer ISR 发送消息到应用层的消息队列，App task 从消息队列中取出消息进行处理。

```

while (true)
{
    if (os_msg_rcv(evt_queue_handle, &event, 0xFFFFFFFF) == true)
    {
        if (event == EVENT_IO_TO_APP)
        {
            T_IO_MSG io_msg;
            if (os_msg_rcv(io_queue_handle, &io_msg, 0) == true)
            {
                app_handle_io_msg(io_msg);
            }
        }
        else
        {
            gap_handle_msg(event);
        }
    }
}
  
```

4. 防丢器 IO 初始化和处理

防丢器定义了 3 个 IO，分别为 LED，KEY，BEEP。在 board.h 中定义如下：

```
#define LED      P0_1          //LED0 ON EVB
#define BEEP     P0_2          //LED1 ON EVB
#define KEY      P2_4
```

4.1. IO 初始化

IO 的初始化流程如下：

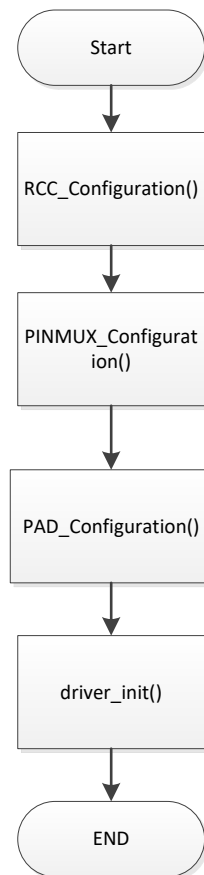


图 4.1 IO 的初始化流程

在 driver_init()中，注册并使能了 KEY 中断，为防止 OS 在初始化中接收到中断导致 OS 逻辑异常，建议 driver_init()放在 apptask 中调用。

```
void app_main_task(void *p_param)
{
    uint8_t event;

    os_msg_queue_create(&io_queue_handle,      MAX_NUMBER_OF_IO_MESSAGE,
    sizeof(T_IO_MSG));
    os_msg_queue_create(&evt_queue_handle, MAX_NUMBER_OF_EVENT_MESSAGE,
    sizeof(uint8_t));
```

```
gap_start_bt_stack(evt_queue_handle,          io_queue_handle,
MAX_NUMBER_OF_GAP_MESSAGE);

driver_init();
.....
.....
.....
}
```

4.2. IO 处理

4.2.1. 按键的处理

防丢器中的按键区分长按和短按，长按和短按的实现如下：

按键的状态定义如下：

```
typedef enum _KeyStatus
{
    keyIdle = 0,
    keyShortPress,
    keyLongPress,
} KeyStatus;
```

当没有按键按下时，按键处于 **keyIdle** 状态（Release）；当有按键按下时，设置按键状态为 **keyShortPress**，翻转中断触发极性，同时开启长按定时器 **os_timer_start(&xTimerLongPress)**。长按定时器 **xTimerLongPress** 定时时间到达后，其回调函数设置按键状态为 **keyLongPress**，同时发出长按的 **message** 给 **apptask**。当按键 **release** 后，中断触发，先翻转中断触发极性（等待后续 **keypress**），然后判断按键状态，如果是 **keyShortPress**，则关闭长按定时器 **os_timer_stop(&xTimerLongPress)**，同时发出短按的 **message** 给 **apptask**；最后设置按键状态为 **keyIdle**。

APP task 收到长按的 **message** 后，进行蓝牙相关的状态切换操作，收到短按的 **message** 后，进行 IO 相关的操作。

防丢器的蓝牙状态如下：

```
PxpStateIdle = 0,
PxpStateAdv = 1,
PxpStateLink = 2,
```

防丢器的 IO 状态如下：

```
IoStateIdle = 0,
IoStateAdvBlink = 1,
IoStateImmAlert = 2,
IoStateLlsAlert = 3,
```

下面是蓝牙状态的迁移图，状态迁移图非常直观的展现了防丢器的蓝牙状态切换逻辑。

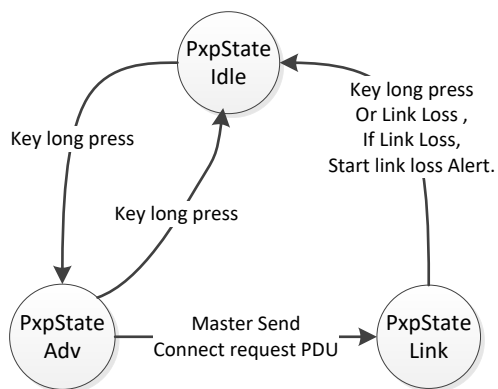


图 4.2 蓝牙状态迁移

防丢器按键短按逻辑:

- A) 如果防丢器在 idle 状态, LED 闪烁一下, 表示设备能够正常工作;
- B) 如果防丢器的 IO 在 Alert 状态, 关闭 LED 和 BEEP;
- C) 如果防丢器在 link 状态, 且 IO 不在 Alert 状态, 则发送 alert 的 notification 到 master。

4.2.2. IO 逻辑的实现

通过函数 void StartPxpIO(uint32_t lowPeroid, uint32_t HighPeroid, uint8_t mode, uint32_t cnt)控制防丢器的 LED 和 BEEP。函数中第一个参数 lowPeroid 表示 IO 翻转时低电平的持续时间, 第二个参数 HighPeroid 表示 IO 翻转时高电平的持续时间, 第三个参数 mode 表示启动 LED 还是启动 BEEP 还是两个同时启动, 最后一个参数 cnt 表示 IO 翻转周期个数。

StartPxpIO 函数通过软件定时器 xTimerPxpIO 实现具体功能, 软件定时器中断处理函数根据当前 IO 的情况设置下一次定时器的定时时间, 并根据 cnt 值判断是否需要重新启动定时器。

5. 广播

防丢器的广播内容如下，手机或者其他蓝牙主机设备进行搜索的时候，会搜索到设备名称为“BLB_PROX”的设备，广播内容包括 GAP_ADTYPE_FLAGS, UUID 为 KNS Service 的 UUID，如进行 active scan 的话，可以扫描到 scan response data 为 KEYRING 的 Appearance 的字段。

```
/** @brief GAP - scan response data (max size = 31 bytes) */
static uint8_t scan_rsp_data[] =
{
    /* BT stack appends Local Name. */
    0x03, /* length */
    0x19, /* type="Appearance" */
    0x40, 0x02, /* Keyring */

    /* place holder for Local Name, filled by BT stack. if not present */
    DEVICE_NAME_LEN, /* length */
    0x09, /* type="Complete local name" */
    DEVICE_NAME /* BLB_PROX */
};

/** @brief GAP - Advertisement data (max size = 31 bytes, best kept short
to conserve power) */
static uint8_t adv_data[] =
{
    /* Core spec. Vol. 3, Part C, Chapter 18 */
    /* Flags */
    0x02, /* length */
    0x01, 0x06, /* type="flags", data="bit 1: LE General Discoverable Mode" */

    /* Adv data encrypt sample */
    0x07, /* length */
    GAP_ADTYPE_MANUFACTURER_SPECIFIC, /* AD Type */
    0x5d, 0x00, /* Company ID */
    'D', 'A', 'T', 'A', /* data */

    /* Service */
    0x11, /* length */
    0x07, /* type="Complete 128-bit UUIDs available" */
    0xA6, 0xF6, 0xF6, 0x07,
    0x4D, 0xC4, 0x9D, 0x98,
    0x6D, 0x45, 0x29, 0xBB,
    0xD0, 0xFF, 0x00, 0x00
};
```

6. GATT 相关 Service 和 Characteristic

防丢器应用中包含如下几个 Service:

1. Immediate Alert Service: 操作防丢器立即开始报警;
2. Link Loss Service: 发生断线时, 防丢器报警;
3. Tx Power Service: 指示发射功率;
4. Battery Service: 汇报设备的电池电量, 提醒更换电池, 电量过低时不可以进行 OTA;
5. Device Information Service: 显示设备基本信息;
6. Key Notification Service: 设置报警次数, 发送按键报警到 Master。

各服务名称及 UUID 如表 6.1 所示。

表 6.1 包含的 Service 以及 UUID 列表

Service Name	Service UUID
Immediate Alert Service	0x1802
Link Loss Service	0x1803
Tx Power Service	0x1804
Battery Service	0x180F
Device Information Service	0x180A
Key Notification Service	0x0000FFD0-BB29-456D-989D-C44D07F6F6A6

6.1. Immediately Alert Service

Immediately Alert Service 是标准的 GATT Service, 所有的功能和定义在 ias.c&ias.h 里实现。

表 6.2 Immediate Alert Service Characteristic 列表

Characteristic Name	Requirement	Characteristic UUID	Properties	Description
Alert Level	M	0x2A06	Write Without Response	See Alert Level

Alert Level 描述了设备的报警级别, 数据格式为 8 位无符号整型, 初始值为 0。报警级别分为 3 级, 分别对应 0--不报警, 1--温和报警, 2--严厉报警, 如表 6.4 所示。

表 6.3 Alert Level Characteristic Value Format

Names	Field Requirement	Format	Mininum Value	Maximum Value	Additional Information
Alert Level	Mandatory	uint8	0	2	See Enumeration

表 6.4 Alert Level Enumerations

Key	0	1	2	3-255
Value	No Alert	Mild Alert	High Level	Reserved

Immediate Alert Service 仅定义一个 characteristic -- Alert Level，该 characteristic 是一个 control point，使对测端可以通过写(write no response)这个 characteristic 来触发本地设备报警，而且通过 Level 的值设定报警的级别。

当本地设备报警被触发后，以下的方式可以停止报警：

- 报警时间到达，目前程序中设定的是 30 次；
- 用户关闭报警；
- 新的 Alert Level 被写入；
- 链路断开，开启新的报警（之前的报警可以理解为停止）。

IAS 的 GATT Attribute Table 如下：

```
const T_ATTRIB_APPL ias_attr_tbl[] =
{
    /*----- Immediate Alert Service -----*/
    {
        (ATTRIB_FLAG_VALUE_INCL | ATTRIB_FLAG_LE), /* wFlags */
        { /* bTypeValue */
            LO_WORD(GATT_UUID_PRIMARY_SERVICE),
            HI_WORD(GATT_UUID_PRIMARY_SERVICE),
            LO_WORD(GATT_UUID_IMMEDIATE_ALERT_SERVICE), /* service UUID */
            HI_WORD(GATT_UUID_IMMEDIATE_ALERT_SERVICE)
        },
        UUID_16BIT_SIZE, /* bValueLen */
        NULL, /* pValueContext */
        GATT_PERM_READ /* wPermissions */
    },

    /* Alert Level Characteristic */
    {
        ATTRIB_FLAG_VALUE_INCL, /* wFlags */
        { /* bTypeValue */
            LO_WORD(GATT_UUID_CHARACTERISTIC),
            HI_WORD(GATT_UUID_CHARACTERISTIC),
            GATT_CHAR_PROP_WRITE_NO_RSP, /* characteristic
properties */
        },
        1, /* bValueLen */
        NULL,
        GATT_PERM_READ /* wPermissions */
    },

    /* Alert Level Characteristic value */
    {
```



```

        ATTRIB_FLAG_VALUE_APPL,                /* wFlags */
        {                                     /* bTypeValue */
            LO_WORD(GATT_UUID_CHAR_ALERT_LEVEL),
            HI_WORD(GATT_UUID_CHAR_ALERT_LEVEL)
        },
        0,                                     /* variable size */
        NULL,
        GATT_PERM_WRITE | GATT_PERM_READ        /*
wPermissions */
    }
};

```

表 6.5 Alert Level Enumerations

Names	Sub Content	Vlaue	Description
Alert Level	msg_type	SERVICE_CALLBACK_TYPE_WRITE_CHAR_VALUE	Write Characteristic Event
	msg_data	write_alert_level	Characteristic Write Value

当对测端通过 IAS 服务向防丢器发送报警数据时，App task 调用 IAS 的写回调函数，并在 app_profile_callback 进行处理。

```

else if (service_id == ias_srv_id)
{
    T_IAS_CALLBACK_DATA *p_ias_cb_data = (T_IAS_CALLBACK_DATA *)p_data;
    APP_PRINT_ERROR0("IAS CallBack.");
    if(p_ias_cb_data->msg_type== SERVICE_CALLBACK_TYPE_WRITE_CHAR_VALUE)
    {
        g_pxp_immediate_alert_level=p_ias_cb_data->msg_data.write_alert_level;
    }
    else
    {
        if (g_pxp_immediate_alert_level == 1)
        {
            gIoState = IoStateImmAlert;
            StopPxpIO();
            StartPxpIO(ALERT_LOW_PERIOD, ALERT_HIGH_PERIOD, LED_BLINK, gTimeParaValue);
        }
        else if (g_pxp_immediate_alert_level == 2)
        {
            gIoState = IoStateImmAlert;
            StartPxpIO(ALERT_LOW_PERIOD, ALERT_HIGH_PERIOD, LED_BLINK | BEEP_ALERT, gTimeParaValue);
        }
        else
        {
            StopPxpIO();
        }
    }
}
}

```

在防丢器中，用户可以发送三种不同等级的 Immediate Alert，即对 IAS Service 写入属性值：

1. 0: 不报警;
2. 1: LED 闪烁报警;
3. 2: LED 闪烁报警和蜂鸣器报警;

6.2. Link Loss Alert Service

Link Loss Alert Service 是标准的 GATT Service,所有的功能和定义在 lls.c&lls.h 里实现。

表 6.6 Link Loss Service Characteristic 列表

Characteristic Name	Requirement	Characteristic UUID	Properties	Description
Alert Level	M	0x2A06	Read/Write	See Alert Level

Alert Level 描述了设备的报警级别,数据格式为 8 位无符号整型,初始值为 0。报警级别分为 3 级,分别对应 0--不报警,1--温和报警,2--严厉报警,如表 6.8 所示。

表 6.7 Alert Level Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Alert Level	Mandatory	uint8	0	2	See Enumeration

表 6.8 Alert Level Enumerations

Key	0	1	2	3-255
Value	No Alert	Mild Alert	High Level	Reserved

Link Loss Alert Service 仅定义一个 characteristic -- Alert Level, 该 characteristic 是读写的,使对测端可以通过写(write response)这个 characteristic 来设置设备的报警级别,报警级别设置好后,当发生 link loss 后,程序会根据设备的报警级别,启动相应的报警状态,包括 LED 闪烁,蜂鸣器报警等。

LLS 的 GATT Attribute Table 如下:

```
const T_ATTRIB_APPL lls_attr_tbl[] =
{
    /*----- Link Loss Service -----*/
    {
        (ATTRIB_FLAG_VALUE_INCL | ATTRIB_FLAG_LE), /* wFlags */
        {
            /* bTypeValue */
            LO_WORD(GATT_UUID_PRIMARY_SERVICE),
            HI_WORD(GATT_UUID_PRIMARY_SERVICE),
            LO_WORD(GATT_UUID_LINK_LOSS_SERVICE), /* service UUID */
            HI_WORD(GATT_UUID_LINK_LOSS_SERVICE)
        },
        UUID_16BIT_SIZE, /* bValueLen */
        NULL, /* pValueContext */
    }
}
```

```

        GATT_PERM_READ                                /* wPermissions */
    },

    /* Alert Level Characteristic */
    {
        ATTRIB_FLAG_VALUE_INCL,                        /* wFlags */
        {                                              /* bTypeValue */
            LO_WORD(GATT_UUID_CHARACTERISTIC),
            HI_WORD(GATT_UUID_CHARACTERISTIC),
            GATT_CHAR_PROP_READ | GATT_CHAR_PROP_WRITE, /* characteristic
properties */
        },
        1,                                             /* bValueLen */
        NULL,
        GATT_PERM_READ                                /* wPermissions */
    },

    /* Alert Level Characteristic value */
    {
        ATTRIB_FLAG_VALUE_APPL,                        /* wFlags */
        {                                              /* bTypeValue */
            LO_WORD(GATT_UUID_CHAR_ALERT_LEVEL),
            HI_WORD(GATT_UUID_CHAR_ALERT_LEVEL)
        },
        0,                                             /* variable size */
        NULL,
        GATT_PERM_READ | GATT_PERM_WRITE              /* wPermissions */
    }
};

```

当对测端通过 LLS 服务向防丢器发送报警级别时，App task 调用 LLS 的写回调函数，并在 `app_profile_callback` 进行处理。

当对测端通过 LLS 服务向防丢器读取报警级别时，App task 调用 LLS 的读回调函数，并在 `app_profile_callback` 进行处理。

```

else if (service_id == lls_srv_id)
{
    T_LLS_CALLBACK_DATA *p_lls_cb_data = (T_LLS_CALLBACK_DATA *)p_data;
    switch (p_lls_cb_data->msg_type)
    {
        case SERVICE_CALLBACK_TYPE_WRITE_CHAR_VALUE:
            g_pxp_linkloss_alert_level =
p_lls_cb_data->msg_data.write_alert_level;
            break;
        case SERVICE_CALLBACK_TYPE_READ_CHAR_VALUE:
            lls_set_parameter(LLS_PARAM_LINK_LOSS_ALERT_LEVEL, 1,
&g_pxp_linkloss_alert_level);
            break;
        default:
            break;
    }
}
}

```

当发生 Link Loss（断线时），调用相应的函数，判断链路丢失的原因，并根据之前设定的报警级别，启动相应的报警，并重新启动广播。

```
case GAP_CONN_STATE_DISCONNECTED:
{
    if ((disc_cause != (HCI_ERR | HCI_ERR_REMOTE_USER_TERMINATE))
        && (disc_cause != (HCI_ERR | HCI_ERR_LOCAL_HOST_TERMINATE)))
    {
        APP_PRINT_ERROR1("app_handle_conn_state_evt: connection lost
cause 0x%x", disc_cause);
    }
    gPxpState = PxpStateIdle;
    if (gPowerFlg == true)
    {
        le_adv_start();
        APP_PRINT_ERROR1("g_pxp_linkloss_alert_level is %d",
g_pxp_linkloss_alert_level);
        gIoState = IoStateLlsAlert;
        if (g_pxp_linkloss_alert_level == 1)
        {
            StartPxpIO(ALERT_LOW_PERIOD, ALERT_HIGH_PERIOD, LED_BLINK,
LL_ASSERT_TIME);
        }
        if (g_pxp_linkloss_alert_level == 2)
        {
            StartPxpIO(ALERT_LOW_PERIOD, ALERT_HIGH_PERIOD, LED_BLINK
| BEEP_ALERT, LL_ASSERT_TIME);
        }
        else
        {
            //nothing to do
        }
    }
}
break;
```

6.3. Tx Power Service

Tx Power Service 是标准的 GATT Service，所有的功能和定义在 tps.c&tps.h 里实现。

Tx Power Service 只包含一个 Tx Power Service Characteristic，为只读的，如表 6.9 所示。

表 6.9 Tx Power Service Characteristic 列表

Characteristic Name	Requirement	Characteristic UUID	Properties	Description
Tx Power Level	M	0x2A07	Read	See Tx Power Level

Tx Power Level 表示当前的发射功率，单位 dBm，范围从-100dBm 至+20dBm，分辨率 为 1dBm。数据格式为有符号 8 位整型，初始值为 0，如表 6.10 所示。

表 6.10 Tx Power Level Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Tx Power Level	Mandatory	sint8	-100	20	none

目前在我们的程序中，默认的 Tx 发射功率是 0dBm。

TPS 的 GATT Attribute Table 如下：

```
const T_ATTRIB_APPL tps_attr_tbl[] =
{
    /*----- TX Power Service -----*/
    {
        (ATTRIB_FLAG_VALUE_INCL | ATTRIB_FLAG_LE), /* wFlags */
        {
            /* bTypeValue */
            LO_WORD(GATT_UUID_PRIMARY_SERVICE),
            HI_WORD(GATT_UUID_PRIMARY_SERVICE),
            LO_WORD(GATT_UUID_TX_POWER_SERVICE), /* service UUID */
            HI_WORD(GATT_UUID_TX_POWER_SERVICE)
        },
        UUID_16BIT_SIZE, /* bValueLen */
        NULL, /* pValueContext */
        GATT_PERM_READ /* wPermissions */
    },

    /* Alert Level Characteristic */
    {
        ATTRIB_FLAG_VALUE_INCL, /* wFlags */
        {
            /* bTypeValue */
            LO_WORD(GATT_UUID_CHARACTERISTIC),
            HI_WORD(GATT_UUID_CHARACTERISTIC),
            GATT_CHAR_PROP_READ, /* characteristic properties */
        },
        1, /* bValueLen */
        NULL,
    }
}
```

```

        GATT_PERM_READ                                /* wPermissions */
    },

    /* Alert Level Characteristic value */
    {
        ATTRIB_FLAG_VALUE_APPL,                        /* wFlags */
        {                                              /* bTypeValue */
            LO_WORD(GATT_UUID_CHAR_TX_LEVEL),
            HI_WORD(GATT_UUID_CHAR_TX_LEVEL)
        },
        0,                                              /* variable size */
        NULL,
        GATT_PERM_READ                                /* wPermissions */
    }
};

```

当对测端通过 TPS 服务向防丢器读取报警级别时，App task 调用 TPS 的读回调函数，并在 `app_profile_callback` 进行处理。

```

else if (service_id == tps_srv_id)
{
    T_TPS_CALLBACK_DATA *p_tps_cb_data = (T_TPS_CALLBACK_DATA *)p_data;
    if (p_tps_cb_data->msg_type == SERVICE_CALLBACK_TYPE_READ_CHAR_VALUE)
    {
        if (p_tps_cb_data->msg_data.read_value_index == TPS_READ_TX_POWER_VALUE)
        {
            uint8_t tps_value = 0;
            tps_set_parameter(TPS_PARAM_TX_POWER, 1, &tps_value);
        }
    }
}

```

6.4. Battery Service

Battery Service 是标准的 GATT Service，所有的功能和定义在 `bas.c&bas.h` 里实现。

Battery Service 包含一个 Battery Level 的 Characteristic，如表 6.11 所示。

表 6.11 Battery Service Characteristic 列表

Characteristic Name	Requirement	Characteristic UUID	Properties	Description
Battery Level	M	0x2A19	Read/Notify	See Battery Level

Battery Level 表示当前电量水平，范围从 0%~100%，数据格式为无符号 8 位整型，如表 6.12 所示。

表 6.12 Battery Level Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information	
Battery Level	Mandatory	uint8	0	100	Enumerations	
					Key	Vlaue
					101-255	Reserved

BAS 的 GATT Attribute Table 如下:

```
static const T_ATTRIB_APPL bas_attr_tbl[] =
{
    /*----- Battery Service -----*/
    /* <<Primary Service>>, .. */
    {
        (ATTRIB_FLAG_VALUE_INCL | ATTRIB_FLAG_LE), /* flags */
        { /* type_value */
            LO_WORD(GATT_UUID_PRIMARY_SERVICE),
            HI_WORD(GATT_UUID_PRIMARY_SERVICE),
            LO_WORD(GATT_UUID_BATTERY), /* service UUID */
            HI_WORD(GATT_UUID_BATTERY)
        },
        UUID_16BIT_SIZE, /* bValueLen */
        NULL, /* p_value_context */
        GATT_PERM_READ /* permissions */
    },

    /* <<Characteristic>>, .. */
    {
        ATTRIB_FLAG_VALUE_INCL, /* flags */
        { /* type_value */
            LO_WORD(GATT_UUID_CHARACTERISTIC),
            HI_WORD(GATT_UUID_CHARACTERISTIC),
#ifdef BAS_BATTERY_LEVEL_NOTIFY_SUPPORT
            (GATT_CHAR_PROP_READ | /* characteristic
properties */
            GATT_CHAR_PROP_NOTIFY)
#else
            GATT_CHAR_PROP_READ
#endif
        }, /* characteristic UUID not needed here, is UUID of next attrib.
        */
        },
        1, /* bValueLen */
        NULL,
        GATT_PERM_READ /* permissions */
    },
    /* Battery Level value */
    {
        ATTRIB_FLAG_VALUE_APPL, /* flags */
        { /* type_value */
            LO_WORD(GATT_UUID_CHAR_BAS_LEVEL),
            HI_WORD(GATT_UUID_CHAR_BAS_LEVEL)
        },
        },
        0, /* bValueLen */
        NULL,
    }
};
```

```

        GATT_PERM_READ                                /* permissions */
    }
    #if BAS_BATTERY_LEVEL_NOTIFY_SUPPORT
    ,
    /* client characteristic configuration */
    {
        ATTRIB_FLAG_VALUE_INCL | ATTRIB_FLAG_CCCD_APPL,                                /*
flags */
        {
                                                    /* type_value */
            LO_WORD(GATT_UUID_CHAR_CLIENT_CONFIG),
            HI_WORD(GATT_UUID_CHAR_CLIENT_CONFIG),
            /* NOTE: this value has an instantiation for each client, a write
to */
            /* this attribute does not modify this default value:
*/
            LO_WORD(GATT_CLIENT_CHAR_CONFIG_DEFAULT), /* client char. config.
bit field */
            HI_WORD(GATT_CLIENT_CHAR_CONFIG_DEFAULT)
        },
        2,                                                    /* bValueLen */
        NULL,
        (GATT_PERM_READ | GATT_PERM_WRITE)                /* permissions */
    }
    #endif
};

```

由于 BAS 的 notify 是可选的属性，所以在 attribute table 中用宏定义进行使能。

当对测端通过 BAS 服务向防丢器读取电量值时，App task 调用 BAS 的读回调函数，并在 app_profile_callback 进行处理。

如果 attribute table 中注册了电量的 notify 属性，当对测端通过 BAS 服务向防丢器写使能电池电量的通知功能，App task 调用 BAS 的 CCCD 更新回调函数，并在 app_profile_callback 进行处理，通知功能打开后，APP 程序可以启动一个定时器，定期上报电池的电量。

```

else if (service_id == bas_srv_id)
{
    T_BAS_CALLBACK_DATA *p_bas_cb_data = (T_BAS_CALLBACK_DATA *)p_data;
    switch (p_bas_cb_data->msg_type)
    {
        case SERVICE_CALLBACK_TYPE_INDIFICATION_NOTIFICATION:
        {
            switch
(p_bas_cb_data->msg_data.notification_indification_index)
            {
                case BAS_NOTIFY_BATTERY_LEVEL_ENABLE:
                {
                    APP_PRINT_INFO0("BAS_NOTIFY_BATTERY_LEVEL_ENABLE");
                }
                break;

                case BAS_NOTIFY_BATTERY_LEVEL_DISABLE:
                {
                    APP_PRINT_INFO0("BAS_NOTIFY_BATTERY_LEVEL_DISABLE");
                }
                break;
                default:

```



```

        break;
    }
}
break;

case SERVICE_CALLBACK_TYPE_READ_CHAR_VALUE:
{
    if (p_bas_cb_data->msg_data.read_value_index ==
BAS_READ_BATTERY_LEVEL)
    {
        uint8_t battery_level = 90;
        APP_PRINT_INFO1("BAS_READ_BATTERY_LEVEL:
battery_level %d", battery_level);
        bas_set_parameter(BAS_PARAM_BATTERY_LEVEL, 1,
&battery_level);
    }
}
break;
default:
break;
}
}

```

6.5. Device Information Service

Device Information Service 是标准的 GATT Service，所有的功能和定义在 dis.c&dis.h 里实现。

Device Information Service 定义了多个只读的 characteristic，而且都是可选的。客户可以通过宏定义进行选择自己所需的 characteristic。

表 6.13 Device Information Service Characteristic 列表

Characteristic Name	Requirement	Characteristic UUID	Properties
Manufacturer Name String	O	0x2A29	Read
Model Number String	O	0x2A24	Read
Serial Number String	O	0x2A25	Read
Hardware Revision String	O	0x2A27	Read
Firmware Revision String	O	0x2A26	Read
Software Revision String	O	0x2A28	Read
System ID	O	0x2A23	Read
Regulatory Certification Data List	O	0x2A2A	Read
PnP ID	O	0x2A50	Read

Device Information Service 中包含一部分显示设备名称和固件版本等基本信息的 Characteristic，如表 6.14 所示。

表 6.14 Device Information Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Manufacturer Name	Mandatory	utf8s	N/A	N/A	none
Model Number	Mandatory	utf8s	N/A	N/A	none
Serial Number	Mandatory	utf8s	N/A	N/A	none
Hardware Revision	Mandatory	utf8s	N/A	N/A	none
Firmware Revision	Mandatory	utf8s	N/A	N/A	none
Software Revision	Mandatory	utf8s	N/A	N/A	none

(1) System ID Characteristic

System ID 由两个字段组成，分别为 40bit 制造商定义的 ID 和 24bit 组织唯一标识符 (OUI)，如表 6.15 所示。

表 6.15 System ID Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Manufacturer Identifier	Mandatory	uint40	0	1099511627775	none
Organization Unique Identifier	Mandatory	uint24	0	16777215	none

(2) IEEE 11073-20601 Regulatory Certification Data List Characteristic

IEEE 11073-20601 Regulatory Certification Data List 列举了设备依附的各种各样的管理或服从认证的项目，如表 6.16 所示。

表 6.16 IEEE 11073-20601 Regulatory Certification Data List Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Data	Mandatory	reg-cert-data-list ^[2]	N/A	N/A	none

(3) PnP ID Characteristic

PnP ID 是一组用于创建唯一设备 ID 的数值，包括了 Vendor ID Source、Vendor ID、Product ID、Product Version，这些数值分别用来辨别具有给定的类型/模板/版本的所有设备，如表 6.17 所示。

表 6.17 PnP ID Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
-------	-------------------	--------	---------------	---------------	------------------------

Vendor ID Source	Mandatory	uint8	1	2	See Enumerations
Vendor ID	Mandatory	uint16	N/A	N/A	None
Product ID	Mandatory	uint16	N/A	N/A	None
Product Version	Mandatory	uint16	N/A	N/A	None

表 6.18 Vendor ID Enumerations

Key	1	2	3-255	0
Value	Bluetooth SIG assigned Company Identifier value from the Assigned	USB Implementer's Forum assigned Vendor ID Value	Reserved for future use	Reserved for future use

DIS 的 GATT Attribute Table 如下:

```
static const T_ATTRIB_APPL dis_attr_tbl[] =
{
    /*----- Device Information Service -----*/
    /* <<Primary Service>> */
    {
        (ATTRIB_FLAG_VALUE_INCL | ATTRIB_FLAG_LE), /* flags */
        { /* type_value */
            LO_WORD(GATT_UUID_PRIMARY_SERVICE),
            HI_WORD(GATT_UUID_PRIMARY_SERVICE),
            LO_WORD(GATT_UUID_DEVICE_INFORMATION_SERVICE), /* service UUID
*/
            HI_WORD(GATT_UUID_DEVICE_INFORMATION_SERVICE)
        },
        UUID_16BIT_SIZE, /* bValueLen */
        NULL, /* p value context */
        GATT_PERM_READ /* permissions */
    }

    #if DIS_CHAR_MANUFACTURER_NAME_SUPPORT
    ,
    /* <<Characteristic>> */
    {
        ATTRIB_FLAG_VALUE_INCL, /* flags */
        { /* type_value */
            LO_WORD(GATT_UUID_CHARACTERISTIC),
            HI_WORD(GATT_UUID_CHARACTERISTIC),
            GATT_CHAR_PROP_READ /* characteristic
properties */
        },
        /* characteristic UUID not needed here, is UUID of next attrib.
*/
        1, /* bValueLen */
        NULL,
        GATT_PERM_READ /* permissions */
    },
    /* Manufacturer Name String characteristic value */

```

```

    {
        ATTRIB_FLAG_VALUE_APPL,                /* flags */
        {
            LO_WORD(GATT_UUID_CHAR_MANUFACTURER_NAME),
            HI_WORD(GATT_UUID_CHAR_MANUFACTURER_NAME)
        },
        0,                                     /* variable size */
        (void *)NULL,
        GATT_PERM_READ                         /* permissions */
    }
#endif

#if DIS_CHAR_MODEL_NUMBER_SUPPORT
,
/* <<Characteristic>> */
{
    ATTRIB_FLAG_VALUE_INCL,                /* flags */
    {
        LO_WORD(GATT_UUID_CHARACTERISTIC),
        HI_WORD(GATT_UUID_CHARACTERISTIC),
        GATT_CHAR_PROP_READ                /* characteristic
properties */
    },
    /* characteristic UUID not needed here, is UUID of next attrib.
*/
    1,                                     /* bValueLen */
    NULL,
    GATT_PERM_READ                         /* permissions */
},
/* Model Number characteristic value */
{
    ATTRIB_FLAG_VALUE_APPL,                /* flags */
    {
        LO_WORD(GATT_UUID_CHAR_MODEL_NUMBER),
        HI_WORD(GATT_UUID_CHAR_MODEL_NUMBER)
    },
    0,                                     /* variable size */
    (void *)NULL,
    GATT_PERM_READ                         /* permissions */
}
#endif

#if DIS_CHAR_SERIAL_NUMBER_SUPPORT
,
/* <<Characteristic>> */
{
    ATTRIB_FLAG_VALUE_INCL,                /* flags */
    {
        LO_WORD(GATT_UUID_CHARACTERISTIC),
        HI_WORD(GATT_UUID_CHARACTERISTIC),
        GATT_CHAR_PROP_READ                /* characteristic properties
*/
    },
    /* characteristic UUID not needed here, is UUID of next attrib.
*/
    1,                                     /* bValueLen */

```

```

        NULL,
        GATT_PERM_READ                                /* permissions */
    },
    /* Serial Number String characteristic value */
    {
        ATTRIB_FLAG_VALUE_APPL,                        /* flags */
        {                                              /* type_value */
            LO_WORD(GATT_UUID_CHAR_SERIAL_NUMBER),
            HI_WORD(GATT_UUID_CHAR_SERIAL_NUMBER)
        },
        0,                                              /* variable size */
        (void *)NULL,
        GATT_PERM_READ                                /* permissions */
    }
#endif

#if DIS_CHAR_HARDWARE_REVISION_SUPPORT
,
/* <<Characteristic>> */
{
    ATTRIB_FLAG_VALUE_INCL,                            /* flags */
    {                                                  /* type_value */
        LO_WORD(GATT_UUID_CHARACTERISTIC),
        HI_WORD(GATT_UUID_CHARACTERISTIC),
        GATT_CHAR_PROP_READ                          /* characteristic properties
*/
        /* characteristic UUID not needed here, is UUID of next attrib.
*/
    },
    1,                                                  /* bValueLen */
    NULL,
    GATT_PERM_READ                                    /* permissions */
},
/* Manufacturer Name String characteristic value */
{
    ATTRIB_FLAG_VALUE_APPL,                            /* flags */
    {                                                  /* type_value */
        LO_WORD(GATT_UUID_CHAR_HARDWARE_REVISION),
        HI_WORD(GATT_UUID_CHAR_HARDWARE_REVISION)
    },
    0,                                                  /* variable size */
    (void *)NULL,
    GATT_PERM_READ                                    /* permissions */
}
#endif

#if DIS_CHAR_FIRMWARE_REVISION_SUPPORT
,
/* <<Characteristic>> */
{
    ATTRIB_FLAG_VALUE_INCL,                            /* flags */
    {                                                  /* type_value */
        LO_WORD(GATT_UUID_CHARACTERISTIC),
        HI_WORD(GATT_UUID_CHARACTERISTIC),
        GATT_CHAR_PROP_READ                          /* characteristic properties
*/

```

```
        /* characteristic UUID not needed here, is UUID of next attrib.
*/
        },
        1, /* bValueLen */
        NULL,
        GATT_PERM_READ /* permissions */
    },
    /* Firmware revision String characteristic value */
    {
        ATTRIB_FLAG_VALUE_APPL, /* flags */
        { /* type_value */
            LO_WORD(GATT_UUID_CHAR_FIRMWARE_REVISION),
            HI_WORD(GATT_UUID_CHAR_FIRMWARE_REVISION)
        },
        0, /* variable size */
        (void *)NULL,
        GATT_PERM_READ /* permissions */
    }
#endif

#if DIS_CHAR_SOFTWARE_REVISION_SUPPORT
,
/* <<Characteristic>> */
{
    ATTRIB_FLAG_VALUE_INCL, /* flags */
    { /* type_value */
        LO_WORD(GATT_UUID_CHARACTERISTIC),
        HI_WORD(GATT_UUID_CHARACTERISTIC),
        GATT_CHAR_PROP_READ /* characteristic properties
*/
        /* characteristic UUID not needed here, is UUID of next attrib.
*/
    },
    1, /* bValueLen */
    NULL,
    GATT_PERM_READ /* permissions */
},
/* Manufacturer Name String characteristic value */
{
    ATTRIB_FLAG_VALUE_APPL, /* flags */
    { /* type_value */
        LO_WORD(GATT_UUID_CHAR_SOFTWARE_REVISION),
        HI_WORD(GATT_UUID_CHAR_SOFTWARE_REVISION)
    },
    0, /* variable size */
    (void *)NULL,
    GATT_PERM_READ /* permissions */
}
#endif

#if DIS_CHAR_SYSTEM_ID_SUPPORT
,
/* <<Characteristic>> */
{
    ATTRIB_FLAG_VALUE_INCL, /* flags */
    { /* type_value */
```

```

        LO_WORD(GATT_UUID_CHARACTERISTIC),
        HI_WORD(GATT_UUID_CHARACTERISTIC),
        GATT_CHAR_PROP_READ                                /* characteristic
properties */
        /* characteristic UUID not needed here, is UUID of next attrib.
*/
    },
    1,                                /* bValueLen */
    NULL,
    GATT_PERM_READ                                /* permissions */
},
/* System ID String characteristic value */
{
    ATTRIB_FLAG_VALUE_APPL,                /* flags */
    {                                    /* type_value */
        LO_WORD(GATT_UUID_CHAR_SYSTEM_ID),
        HI_WORD(GATT_UUID_CHAR_SYSTEM_ID)
    },
    0,                                /* variable size */
    (void *)NULL,
    GATT_PERM_READ                                /* permissions */
}
#endif

#if DIS_CHAR_IEEE_CERTIF_DATA_LIST_SUPPORT
,

/* <<Characteristic>> */
{
    ATTRIB_FLAG_VALUE_INCL,                /* flags */
    {                                    /* type_value */
        LO_WORD(GATT_UUID_CHARACTERISTIC),
        HI_WORD(GATT_UUID_CHARACTERISTIC),
        GATT_CHAR_PROP_READ                                /* characteristic properties
*/
        /* characteristic UUID not needed here, is UUID of next attrib.
*/
    },
    1,                                /* bValueLen */
    NULL,
    GATT_PERM_READ                                /* permissions */
},
/* Manufacturer Name String characteristic value */
{
    ATTRIB_FLAG_VALUE_APPL,                /* flags */
    {                                    /* type_value */
        LO_WORD(GATT_UUID_CHAR_IEEE_CERTIF_DATA_LIST),
        HI_WORD(GATT_UUID_CHAR_IEEE_CERTIF_DATA_LIST)
    },
    0,                                /* variable size */
    (void *)NULL,
    GATT_PERM_READ                                /* permissions */
}
#endif

#if DIS_CHAR_PNP_ID_SUPPORT

```

```

,
/* <<Characteristic>> */
{
    ATTRIB_FLAG_VALUE_INCL,          /* flags */
    {                                /* type_value */
        LO_WORD(GATT_UUID_CHARACTERISTIC),
        HI_WORD(GATT_UUID_CHARACTERISTIC),
        GATT_CHAR_PROP_READ          /* characteristic properties */
    },
    /* characteristic UUID not needed here, is UUID of next attrib. */
    /*
    1,                                /* bValueLen */
    NULL,
    GATT_PERM_READ                    /* permissions */
    },
    /* Manufacturer Name String characteristic value */
    {
        ATTRIB_FLAG_VALUE_APPL,      /* flags */
        {                            /* type_value */
            LO_WORD(GATT_UUID_CHAR_PNP_ID),
            HI_WORD(GATT_UUID_CHAR_PNP_ID)
        },
        0,                            /* variable size */
        (void *)NULL,
        GATT_PERM_READ                /* permissions */
    }
}
#endif
};

```

当对测端通过 DIS 服务向防丢器读取设备信息时，App task 调用 DIS 的读回调函数，并在 `app_profile_callback` 进行处理。

```

else if (service_id == dis_srv_id)
{
    T_DIS_CALLBACK_DATA *p_dis_cb_data = (T_DIS_CALLBACK_DATA *)p_data;
    switch (p_dis_cb_data->msg_type)
    {
        case SERVICE_CALLBACK_TYPE_READ_CHAR_VALUE:
        {
            if (p_dis_cb_data->msg_data.read_value_index ==
DIS_READ_MANU_NAME_INDEX)
            {
                const uint8_t DISManufacturerName[] = "Realtek BT";
                dis_set_parameter(DIS_PARAM_MANUFACTURER_NAME,
                                sizeof(DISManufacturerName),
                                (void *)DISManufacturerName);
            }
            else if (p_dis_cb_data->msg_data.read_value_index ==
DIS_READ_MODEL_NUM_INDEX)
            {
                const uint8_t DISModelNumber[] = "Model Nbr 0.9";
                dis_set_parameter(DIS_PARAM_MODEL_NUMBER,
                                sizeof(DISModelNumber),
                                (void *)DISModelNumber);
            }
        }
    }
}

```



```

        else if (p_dis_cb_data->msg_data.read_value_index ==
DIS_READ_SERIAL_NUM_INDEX)
        {
            const uint8_t DISSerialNumber[] = "RTKBeeSerialNum";
            dis_set_parameter(DIS_PARAM_SERIAL_NUMBER,
                            sizeof(DISSerialNumber),
                            (void *)DISSerialNumber);
        }
        else if (p_dis_cb_data->msg_data.read_value_index ==
DIS_READ_HARDWARE_REV_INDEX)
        {
            const uint8_t DISHardwareRev[] = "RTKBeeHardwareRev";
            dis_set_parameter(DIS_PARAM_HARDWARE_REVISION,
                            sizeof(DISHardwareRev),
                            (void *)DISHardwareRev);
        }
        else if (p_dis_cb_data->msg_data.read_value_index ==
DIS_READ_FIRMWARE_REV_INDEX)
        {
            const uint8_t DISFirmwareRev[] = "RTKBeeFirmwareRev";
            dis_set_parameter(DIS_PARAM_FIRMWARE_REVISION,
                            sizeof(DISFirmwareRev),
                            (void *)DISFirmwareRev);
        }
        else if (p_dis_cb_data->msg_data.read_value_index ==
DIS_READ_SOFTWARE_REV_INDEX)
        {
            const uint8_t DISSoftwareRev[] = "RTKBeeSoftwareRev";
            dis_set_parameter(DIS_PARAM_SOFTWARE_REVISION,
                            sizeof(DISSoftwareRev),
                            (void *)DISSoftwareRev);
        }
        else if (p_dis_cb_data->msg_data.read_value_index ==
DIS_READ_SYSTEM_ID_INDEX)
        {
            const uint8_t DISSystemID[DIS_SYSTEM_ID_LENGTH] = {0, 1, 2,
0, 0, 3, 4, 5};
            dis_set_parameter(DIS_PARAM_SYSTEM_ID,
                            sizeof(DISSystemID),
                            (void *)DISSystemID);
        }
        else if (p_dis_cb_data->msg_data.read_value_index ==
DIS_READ_IEEE_CERT_STR_INDEX)
        {
            const uint8_t DISIEEEDataList[] = "RTKBeeIEEEDatalist";
            dis_set_parameter(DIS_PARAM_IEEE_DATA_LIST,
                            sizeof(DISIEEEDataList),
                            (void *)DISIEEEDataList);
        }
        else if (p_dis_cb_data->msg_data.read_value_index ==
DIS_READ_PNP_ID_INDEX)
        {
            uint8_t DISPnpID[DIS_PNP_ID_LENGTH] = {0};
            dis_set_parameter(DIS_PARAM_PNP_ID,

```

```

        sizeof(DISPNPID),
        DISPNPID);
    }

    }
    break;
default:
    break;
}
}

```

6.6. Key Notification Service

Key Notification Service 是自定义的 GATT Service，是防丢器私有的，所有的功能和定义在 `kns.c&kns.h` 里实现。

Key Notification Service 定义了两个 characteristic，一个为可读写的，用来配置 link loss 后和 immediately alert 的报警次数，如表 6.19 所示。

另外一个为通知属性，用来向 master 发送 alert 消息，如表 6.19 所示。

表 6.19 Key Notification Service Characteristic 列表

Characteristic Name	Requirement	Characteristic UUID	Properties	Description
Set Alert time	M	0x0000FFD1-BB29-456D-989D-C44D07F6F6A6	Read/Write	See Set Alert Level
Key Value	M	0x0000FFD2-BB29-456D-989D-C44D07F6F6A6	Notify	See Key Value

Set Alert Time Characteristic

Set Alert Time 为自定义的设置时间的 Characteristic。数据格式为 32 位无符号整型，初始值为 30，单位为秒，最小值为 0，最大值为 0xffffffff，如表 6.20 所示。

表 6.20 Set Alert Time Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Set Alert Time	Mandatory	uint32	0	0xffffffff	none

Key Value Characteristic

Key Value 表示按键信息，数据格式为 8 位无符号整型。在连线的环境下按下发送 value 1 到 master 端，如表 6.21 所示。

表 6.21 Key Value Characteristic Value Format

Names	Field Requirement	Format	Value	Additional Information
Key Value	Mandatory	uint8	0	None

KNS 的 GATT Attribute Table 如下:

```
static const T_ATTRIB_APPL kns_attr_tbl[] =
{
    /*----- simple key Service -----*/
    /* <<Primary Service>>, .. */
    {
        (ATTRIB_FLAG_VOID | ATTRIB_FLAG_LE), /* wFlags */
        { /* bTypeValue */
            LO_WORD(GATT_UUID_PRIMARY_SERVICE),
            HI_WORD(GATT_UUID_PRIMARY_SERVICE),
        },
        UUID_128BIT_SIZE, /* bValueLen */
        (void *)GATT_UUID128_KNS_SERVICE, /* pValueContext */
        GATT_PERM_READ /* wPermissions */
    },

    /* Set para Characteristic */
    {
        ATTRIB_FLAG_VALUE_INCL, /* wFlags */
        { /* bTypeValue */
            LO_WORD(GATT_UUID_CHARACTERISTIC),
            HI_WORD(GATT_UUID_CHARACTERISTIC),
            GATT_CHAR_PROP_READ | GATT_CHAR_PROP_WRITE, /*
characteristic properties */
        },
        1, /* bValueLen */
        NULL,
        GATT_PERM_READ /* wPermissions */
    },

    /* Set para Characteristic value */
    {
        ATTRIB_FLAG_VALUE_APPL | ATTRIB_FLAG_UUID_128BIT, /*
wFlags */
        { /* bTypeValue */
            GATT_UUID128_CHAR_PARAM
        },
        0, /* variable size */
        NULL,
        GATT_PERM_READ | GATT_PERM_WRITE /*
wPermissions */
    },

    /* Key <<Characteristic>>, .. */
    {
        ATTRIB_FLAG_VALUE_INCL, /* wFlags */
        { /* bTypeValue */
            LO_WORD(GATT_UUID_CHARACTERISTIC),
            HI_WORD(GATT_UUID_CHARACTERISTIC),
            ( /* characteristic properties */
                GATT_CHAR_PROP_NOTIFY)
        }
    }
}
```

```

        /* characteristic UUID not needed here, is UUID of next attrib.
*/
        },
        1, /* bValueLen */
        NULL,
        GATT_PERM_READ /* wPermissions */
    },
    /* simple key value */
    {
        ATTRIB_FLAG_VALUE_APPL | ATTRIB_FLAG_UUID_128BIT, /*
wFlags */
        { /* bTypeValue */
            GATT_UUID128_CHAR_KEY
        },
        0, /* bValueLen */
        NULL,
        GATT_PERM_READ /* wPermissions */
    },
    /* client characteristic configuration */
    {
        ATTRIB_FLAG_VALUE_INCL | ATTRIB_FLAG_CCCD_APPL, /*
wFlags */
        { /* bTypeValue */
            LO_WORD(GATT_UUID_CHAR_CLIENT_CONFIG),
            HI_WORD(GATT_UUID_CHAR_CLIENT_CONFIG),
            /* NOTE: this value has an instantiation for each client, a write
to */
            /* this attribute does not modify this default value:
*/
            LO_WORD(GATT_CLIENT_CHAR_CONFIG_DEFAULT), /* client char. config.
bit field */
            HI_WORD(GATT_CLIENT_CHAR_CONFIG_DEFAULT)
        },
        2, /* bValueLen */
        NULL,
        (GATT_PERM_READ | GATT_PERM_WRITE) /* wPermissions */
    }
};

```

当对测端通过 KNS 服务向防丢器读/写参数时（link loss and immediately alert 报警的次數），App task 调用 KNS 的读/写回调函数，并在 app_profile_callback 进行处理。

当对测端通过 KNS 服务向防丢器写使能按键报警的通知功能，App task 调用 KNS 的 CCCD 更新回调函数，并在 app_profile_callback 进行处理，通知功能打开后，在连接状态时，用户短按按键，防丢器向 master 发送报警的 notification。

```

else if (service_id == kns_srv_id)
{
    T_KNS_CALLBACK_DATA *p_kns_cb_data = (T_KNS_CALLBACK_DATA
*)p_data;
    switch (p_kns_cb_data->msg_type)
    {
        case SERVICE_CALLBACK_TYPE_INDIFICATION_NOTIFICATION:
        {
            switch
            (p_kns_cb_data->msg_data.notification_indification_index)

```

```
{
    case KNS_NOTIFY_ENABLE:
    {
        APP_PRINT_INFO0("KNS_NOTIFY_ENABLE");
    }
    break;

    case KNS_NOTIFY_DISABLE:
    {
        APP_PRINT_INFO0("KNS_NOTIFY_DISABLE");
    }
    break;
    default:
    break;
}
break;

case SERVICE_CALLBACK_TYPE_READ_CHAR_VALUE:
{
    if (p_kns_cb_data->msg_data.read_index == KNS_READ_PARA)
    {
        APP_PRINT_INFO0("KNS_READ_PARA");
        kns_set_parameter(KNS_PARAM_VALUE, 4,
&gTimeParaValue);
    }
}
break;
case SERVICE_CALLBACK_TYPE_WRITE_CHAR_VALUE:
{
    APP_PRINT_INFO1("KNS_WRITE_PARA%x", p_kns_cb_data->msg_data.write_value);
    gTimeParaValue = p_kns_cb_data->msg_data.write_value;
}
break;

default:
break;
}
}
```

7. 服务的初始化和注册回调函数

防丢器一共有 6 个 service，在 main 函数中通过调用 `app_le_profile_init()` 进行注册和初始化。服务的注册和初始化如下：

```
void app_le_profile_init(void)
{
    server_init(6);
    ias_srv_id = ias_add_service(app_profile_callback);
    lls_srv_id = lls_add_service(app_profile_callback);
    tps_srv_id = tps_add_service(app_profile_callback);
    kns_srv_id = kns_add_service(app_profile_callback);
    bas_srv_id = bas_add_service(app_profile_callback);
    dis_srv_id = dis_add_service(app_profile_callback);
    server_register_app_cb(app_profile_callback);
}
```

8. DLPS

8.1 DLPS 概述

RTL8762E 支持 DLPS (Deep Lower Power State, 即深度睡眠状态) 模式^[4], 当系统多数时间处于空闲状态时, 进入该模式可以大大减少功耗。该模式下 Power、Clock、CPU、Peripheral、RAM 都可以关闭 (掉电) 以降低系统的功耗, 进入 DLPS 模式之前需要保存必要的数。当有事件需要处理时, 系统就会退出 DLPS 模式, CPU、Peripheral、Clock、RAM 重新上电并且恢复到进入 DLPS 之前的状态, 然后响应唤醒事件。

8.2 DLPS 使能和配置

打开 DLPS 的配置有以下步骤:

(1). 打开 board.h 中的相关宏:

```
#define DLPS_EN 1
```

对程序中所使用的相关模块, 还需要打开各模块的宏。

```
#define USE_USER_DEFINE_DLPS_EXIT_CB 1
```

```
#define USE_USER_DEFINE_DLPS_ENTER_CB 1
```

```
#define USE_GPIO_DLPS 1
```

(2). 在 main.c 里的 pwr_mgr_init 函数中注册 DLPS 相关 Callback function:

a). DLPS_IORegUserDlpsEnterCb: 在 callback function 中执行进入 DLPS 时所需的配置;

b). DLPS_IORegUserDlpsExitCb: 在 callback function 中执行退出 DLPS 时所需的配置;

c). dlps_check_cb_reg: 通过配置注册 DLPS_PxpCheck 后, 通过 allowedPxpEnterDlps 的值设置防丢器是否可以进入 DLPS。

```
void PxpEnterDlpsSet(void)
{
    Pad_Config(KEY, PAD_SW_MODE, PAD_IS_PWRON, PAD_PULL_UP, PAD_OUT_DISABLE,
PAD_OUT_LOW);
    Pad_Config(LED, PAD_SW_MODE, PAD_IS_PWRON, PAD_PULL_DOWN,
PAD_OUT_DISABLE, PAD_OUT_LOW);
    Pad_Config(BEEP, PAD_SW_MODE, PAD_IS_PWRON, PAD_PULL_DOWN,
PAD_OUT_DISABLE, PAD_OUT_LOW);
    System_WakeUpDebounceTime(0x8);
    if (keystatus)
    {
        System_WakeUpPinEnable(KEY, PAD_WAKEUP_POL_LOW,
PAD_WK_DEBOUNCE_ENABLE);
    }
    else
    {
        System_WakeUpPinEnable(KEY, PAD_WAKEUP_POL_HIGH,
```

```

PAD_WK_DEBOUNCE_ENABLE);
    }

    #if (AON_WDG_ENABLE == 1)
        aon_wdg_enable();
    #endif
}

void PxpExitDlpsInit(void)
{
    Pad_Config(LED,      PAD_PINMUX_MODE,      PAD_IS_PWRON,      PAD_PULL_NONE,
PAD_OUT_ENABLE, PAD_OUT_LOW);
    Pad_Config(BEEP,      PAD_PINMUX_MODE,      PAD_IS_PWRON,      PAD_PULL_NONE,
PAD_OUT_ENABLE, PAD_OUT_LOW);
    Pad_Config(KEY,      PAD_PINMUX_MODE,      PAD_IS_PWRON,      PAD_PULL_UP,
PAD_OUT_DISABLE, PAD_OUT_LOW);

    #if (AON_WDG_ENABLE == 1)
        aon_wdg_disable();
    #endif
}

bool DLPS_PxpCheck(void)
{
    return allowedPxpEnterDlps;
}

void pwr_mgr_init(void)
{
    #if DLPS_EN
        if (false == dlps_check_cb_reg(DLPS_PxpCheck))
        {
            DBG_DIRECT("Error: dlps_check_cb_reg(DLPS_RcuCheck) failed!\n");
        }
        DLPS_IORegUserDlpsEnterCb(PxpEnterDlpsSet);
        DLPS_IORegUserDlpsExitCb(PxpExitDlpsInit);
        DLPS_IORegister();
        lps_mode_set(PLATFORM_DLPS_PFM);
    #endif
}

```

8.3 DLPS 的条件和唤醒源

防丢器应用中，广播状态和连接状态均可进入 DLPS，但需要符合相关广播参数及连接参数的设置。

(1). 广播状态：广播参数符合条件并且开启 DLPS 功能时，系统可以直接进入 DLPS。在需要广播时，系统自动退出 DLPS，发送广播包，随后再次进入 DLPS。

(2). 连接状态：当防丢器与对测端建立连接之后，防丢器端会请求符合 DLPS 的连接参数更新。当参数更新成功且开启 DLPS 时，便可以进入 DLPS。为保证防丢器能够正确进入 DLPS，需要请求更改连接参数，连接参数更新函数如下：ChangeConnectionParameter(400,0,

2000); //interval = 400*1.25ms

```
void ChangeConnectionParameter(uint16_t interval, uint16_t latency, uint16_t timeout)
{
    le_update_conn_param(0, interval, interval, latency, timeout / 10, interval * 2 - 2,
                        interval * 2 - 2);
}
```

可以通过蓝牙事件、软件定时器、RTC 以及 Wakeup Pin 将防丢器从 DLPS 中唤醒。

防丢器的按键需要选择有 Wakeup 功能的 Pin，否则按键功能可能不生效。用户在处理按键唤醒事件时，应在唤醒并进入 GPIO 中断后通过 `allowedPxpEnterDlps` 暂时禁止系统进入 DLPS，待按键消息处理完成之后再允许进入，保证按键事件不被影响。

9. 参考文献

- [1] RTL8762C PXP Design Spec.pdf
- [2] IEEE Std 11073-20601™- 2008 Health Information – Personal Health Device Communication
– Application Profile – Optimized Exchange Protocol – version 1.0 or later.
- [3] Profile Interface Design.pdf